

Assignment 2

```
In [1]: import numpy as np
        from scipy.special import softmax
```

Problem 1

a: NLP

Please discuss the recent trend of rapidly increasing sizes of NLP architectures.

The recent trends in Natural Language Processing (NLP) have changed the way we use the computer. Chatbots are getting more and more popular, and we (users) expect them to only get better. To improve the chatbot, it comes with a cost of size. The network is becoming larger, and this might be a problem. At the moment (DEC2023) a chatgpt query cost 3Wh which translates to 36 cents, according to Medium. This is around 10 times more than a google search. Chatgpt alone consumes 0.5 million KWh pr day. In 2024, where minimizing power consumption is a huge deal, we need to consider how much better the NLP gets, when the increases in size and therefore consume more power.

b: Transfer Learning

Classify the following example of transfer learning. More exactly, what are the domains and tasks, and which are being changed?

Source: Using a step counter to monitor exercise in healthy people.

Target: Using a step counter to indicate recovery progression in a patient

When talking about transfer learning, we often use the use the terminology of domain and task, and source and target.

In this case where we want to use a step counter for two different purposes, we can say that the domain is the same $D_S = D_T$, since we use a step counter in both scenarios, but the tasks are different.

This is an example of inductive transfer learning.

c: Attention

Assume dotproduct attention, and that the hidden states of the encoder layer are $[0,1,4],[-1,1,2],[1,1,1],[2,1,1]$. If the activation for the previous decoder is $[0.1,1,-2]$, what is the attention-context vector?

```
In [2]: # Encoder hidden states
        encoder_hidden_states = np.array([
            [0, 1, 4],
            [-1, 1, 2],
            [1, 1, 1],
            [2, 1, 1]
        ])
```

```

# Decoder activation (s_(i-1))
decoder_activation = np.array([0.1, 1, -2])

# Step 1: Calculate attention scores using dot product (a_ij = s_(i-1)^T * h_j)
scores = np.dot(encoder_hidden_states, decoder_activation)
print(f"a_ij = {scores}")

# Step 2: Apply softmax to get attention weights (alpha_ij = softmax(a_ij))
attention_weights = softmax(scores)
print(f"alpha_ij = {attention_weights}")

# Step 3: Calculate attention-context vector as weighted sum of encoder hidden states (c_i)
attention_context_vector = np.sum(attention_weights[:, np.newaxis] * encoder_hidden_states, axis=1)
print(f"c_i = {attention_context_vector}")

```

```

a_ij = [-7.  -3.1 -0.9 -0.8]
alpha_ij = [0.00101111 0.04995145 0.45081253 0.4982249 ]
c_i = [1.39731088 1.         1.05298479]

```

The attention context vector is approx: $c_i = [1.4, 1, 1.05]$

d: Transformers

Explain the 'positional encoding' step for transformers. Why is it done, how is it done?

Positional encoding is an important step in transformer models, which are used in natural language processing tasks.

Transformers do not have a built-in sense of the order of the input tokens because they process the input sequence in parallel rather than sequentially. Because of this, positional encoding is introduced. The positional encoding provides the order information, allowing the model to understand and remember the order of the tokens.

Positional encoding involves adding a set of vectors to the inputs, where each vector represents the position of a token in the sequence. These vectors are designed to encode positional information in a way that the model can learn to interpret.

A common method for generating positional encodings is using sine and cosine functions of different frequencies.

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

Where pos is the position in the sequence and i is the dimensions index.

The input to the transformer becomes:

$$X \rightarrow X + PE$$

e: Bounding box detection:

Given a dataset with two classes; cats and dogs, and the following detections:

TP = True positive

FP = False positive

cat_det = [TP, FP, TP, FP, TP]

pred_scores_cat = [0.7, 0.3, 0.5, 0.6, 0.5]

dog_det = [FP, TP, TP, FP, TP, TP]

pred_scores_dog = [0.4, 0.3, 0.9, 0.5, 0.6, 0.7]

There are in total 3 cats and 4 dogs in the images.

Calculate the mean average precision (mAP)

We use the procedure from slides and the formula from slides:

- First we sort by pred_score
- Then we calculate precision and recall
- Then we calculate the average precision with the formula:

$$AP = \sum_{n=1} (R_{n+1} - R_n) P(R_{n+1})$$

- The mean average is then the mean between the two average precisions

```
In [3]: # Given data
cat_det = ['TP', 'FP', 'TP', 'FP', 'TP']
pred_scores_cat = [0.7, 0.3, 0.5, 0.6, 0.5]
dog_det = ['FP', 'TP', 'TP', 'FP', 'TP', 'TP']
pred_scores_dog = [0.4, 0.3, 0.9, 0.5, 0.6, 0.7]

def precision_recall(detection, prediction):
    # Sort the predictions by decreasing score
    sorted_indices = np.argsort(prediction)[::-1]
    detection = np.array(detection)[sorted_indices]

    # Calculate precision and recall
    tp = np.cumsum(detection == 'TP')
    fp = np.cumsum(detection == 'FP')
    # Count the number of true positives
    len_true = np.sum(detection == 'TP')
    # Calculate precision and recall
    precision = tp / (tp + fp)
    recall = tp / len_true
    # Calculate average precision
    ap = 0
    for i in range(len(recall) - 1):
        ap += (recall[i+1] - recall[i]) * precision[i+1]
    return precision, recall, ap

# Calculate precision, recall and average precision for cat detections
precision_cat, recall_cat, ap_cat = precision_recall(cat_det, pred_scores_cat)
print(f"Precision cat: {precision_cat}")
```

```

print(f"Recall cat: {recall_cat}")
print(f"Average Precision cat: {np.round(ap_cat,2)}")

# Calculate precision, recall and average precision for dog detections
precision_dog, recall_dog, ap_dog = precision_recall(dog_det, pred_scores_dog)
print(f"Precision dog: {precision_dog}")
print(f"Recall dog: {recall_dog}")
print(f"Average Precision dog: {np.round(ap_dog,2)}")

# Calculate mean average precision
map = (ap_cat + ap_dog) / 2
print(f"Mean Average Precision: {np.round(map,2)}")

```

```

Precision cat: [1.          0.5          0.66666667 0.75          0.6          ]
Recall cat: [0.33333333 0.33333333 0.66666667 1.          1.          ]
Average Precision cat: 0.47
Precision dog: [1.          1.          1.          0.75          0.6          0.66666667]
Recall dog: [0.25 0.5  0.75 0.75 0.75 1.   ]
Average Precision dog: 0.67
Mean Average Precision: 0.57

```

f: Semantic segmentation - FCN 1:

Given an image sized 1024x768x3 (width x height x channels), with 7 classes. What is the size of the target image if targets are one-hot encoded?

In semantic segmentation, each pixel needs to be classified. After this segmentation, we are able to create an 1024x768x1 image with the most probable classes for each pixel.

Since each pixel has a probability to belong in each of the 7 classes, the target needs to be 1024x768xclasses = 1024x768x7.

g: Semantic segmentation - FCN 2:

What is a fully-convolutional network? When is it useful?

Like the name states, a fully-convolutional network (FCN) is a neural network that consists of only convolutional layers. This type of network is commonly used in computer vision tasks, such as image segmentation, where the input and output are images.

It is useful when you want to classify each pixel of an image. This is useful for many applications, where you want to divide your image into multiple segments. Eg. dividing a picture into its core components

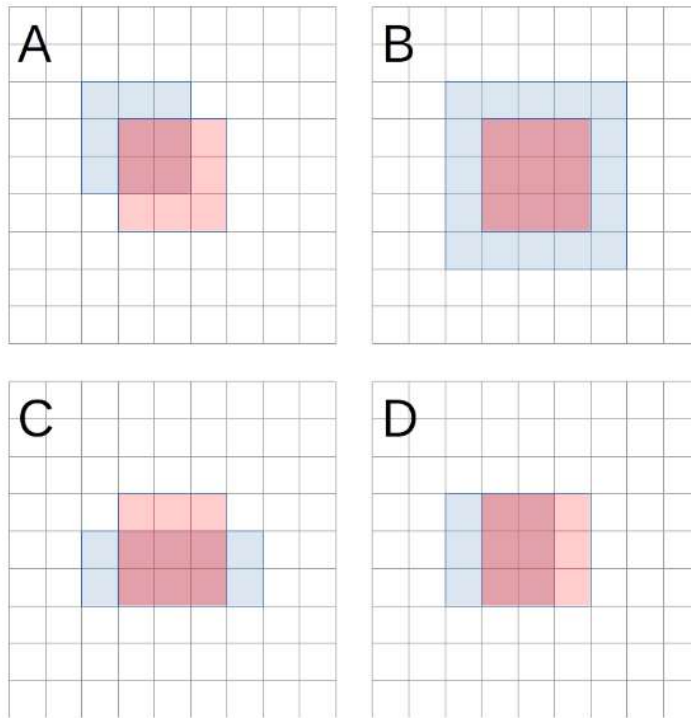
h: Residual Networks:

Explain residual layers and their advantage.

Residual layers are fundamental components of Residual Networks (ResNets), which are designed to address the vanishing gradient problem, making it easier to train very deep neural networks. These layers incorporate skip connections that add the input directly to the output, facilitating better gradient flow. This approach not only mitigates the vanishing gradient issue but also enables faster convergence during training, leading to higher accuracy on various tasks.

i: Intersection-Over-Union

Calculate the intersection over union in for these four bounding-boxes and target bounding boxes:



Intersection over Union (IoU) is measure of how good a bounding box fits an object

The formula for IoU is given by:

$$IOU = \frac{\text{Area of Intersection of two boxes}}{\text{Area of Union of two boxes}}$$

Using this formula we can calculate the IoU of two boxes.

$$IoU_A = \frac{4}{2 \cdot 9 - 4} = \frac{4}{14} = \frac{2}{7}$$

$$IoU_B = \frac{9}{5 \cdot 5} = \frac{9}{25}$$

$$IoU_C = \frac{6}{10 + 3} = \frac{6}{13}$$

$$IoU_D = \frac{6}{2 \cdot 6} = \frac{6}{12} = \frac{1}{2}$$

j: Variational autoencoders:

What are the strengths of a variational autoencoder (VAE) compared to an autoencoder (AE)?

Both VAE and AE are encoders used for data compression and feature extraction. They work somehow quite different. The main difference between the two encoders lies in the regularization of the latent space, where the compressed data is stored. VAEs regularize the latent space, AE does not. This has the advantage that VAE "rethink" the latent space which turn out to be quite an advantages when generating new data samples. The reason for this, is that the latent space gets more well-organized, which makes it easier to generate new similar data.

Problem 2

In this problem, you will build a waveform detector for 1D time series. Below is shared a piece of code defining a (train) dataset and a test set. As you will see when you open the file, the data is synthetic - a waveform is combined with a background 'noise' signal, allowing virtually infinite training data.

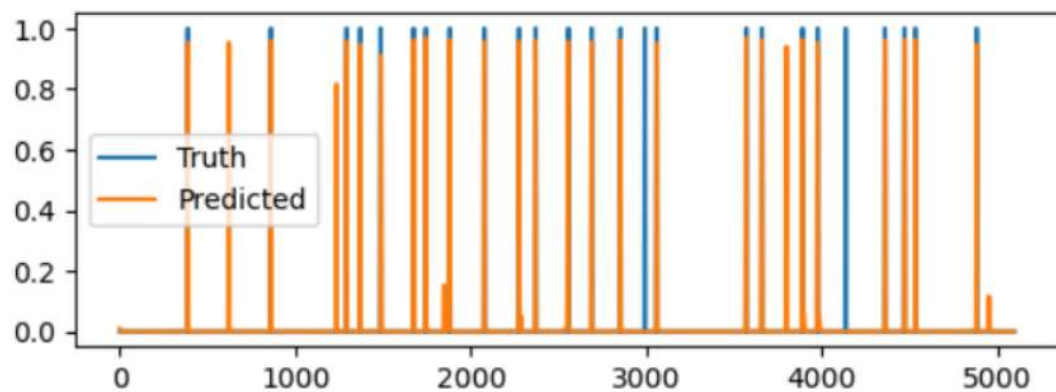
The task is:

Using the dataset supplied below, create a neural network which can detect the ends of waveforms (marked by '1' in the labels).

You are free to use either GRU, LSTM, xLSTM, state space models or Transformers to do the task.

Dataset generation: [data_setup](#)

use the 'waveformLoader' as data loader for your training data, and 'testX,testy' as the test data. Report your performance as an average loss over the test data and with a plot of 'testy' and 'net(testX)' as a function of time, such as:



```
In [4]: ### Load packages

import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn

###

#Prepare waveform data set container:
class waveFormDataset(torch.utils.data.Dataset):
    """waveFormDetectionDataset"""

    def __init__(self, waveforms, backgroundGenerator, targetRate=1):
        """
        Args:
            waveforms: array of size nxl, showing n different examples of target waveforms
            backgroundGenerator: function, no input, outputs non-target signals, length L:
        """

        if len(waveforms.shape)==0:
            waveforms=np.expand_dims(waveforms,0)

        self.ntargets=waveforms.shape[0]
```

```

        self.targets=waveforms
        self.backgroundGenerator=backgroundGenerator
        self.targetRate=targetRate

    def __len__(self):
        return 1000 #arbitrary

    def __getitem__(self, idx):
        #combines targets and backgroundGenerator. includes target in targetRate*100 percent

        sample=self.backgroundGenerator()

        assert(len(sample)>=self.targets.shape[1])

        labels=sample*0

        if np.random.rand()<=self.targetRate:
            target=self.targets[np.random.randint(0,self.ntargets),:]
            target=torch.reshape(target,(-1,1))
            targetL=len(target)

            backgroundL=len(sample)

            endBuffer=4 #we need to ensure a bit of space at the end for placing 1-labels
            startIdx=np.random.randint(0,backgroundL-targetL-endBuffer,1)[0]
            sample[startIdx:(startIdx+targetL)]=sample[startIdx:(startIdx+targetL)]+target
            labels[startIdx+targetL+range(0,endBuffer)]=1

        return sample,labels

# %%

# setup dataset

#training:
targetL=30
waveform=np.sin(np.linspace(0,np.pi*3,targetL))/np.linspace(.2,5,targetL)
waveform=np.expand_dims(waveform,0)
waveform=torch.tensor(waveform, dtype=torch.float32)

def bgg():
    backgroundL=100
    scale=.5
    background=torch.tensor(np.random.normal(0,scale=scale,size=(backgroundL,1)),dtype=torch.float32)
    return background

wfDataset=waveFormDataset(waveform,bgg,.5)
e=wfDataset.__getitem__(1)
waveformLoader=torch.utils.data.DataLoader(wfDataset,batch_size=32)
# %%

#seed random number generator:
torch.manual_seed(1066)
np.random.seed(1066)

#test dataset:
nTest=50
testX,testy=wfDataset.__getitem__(1)

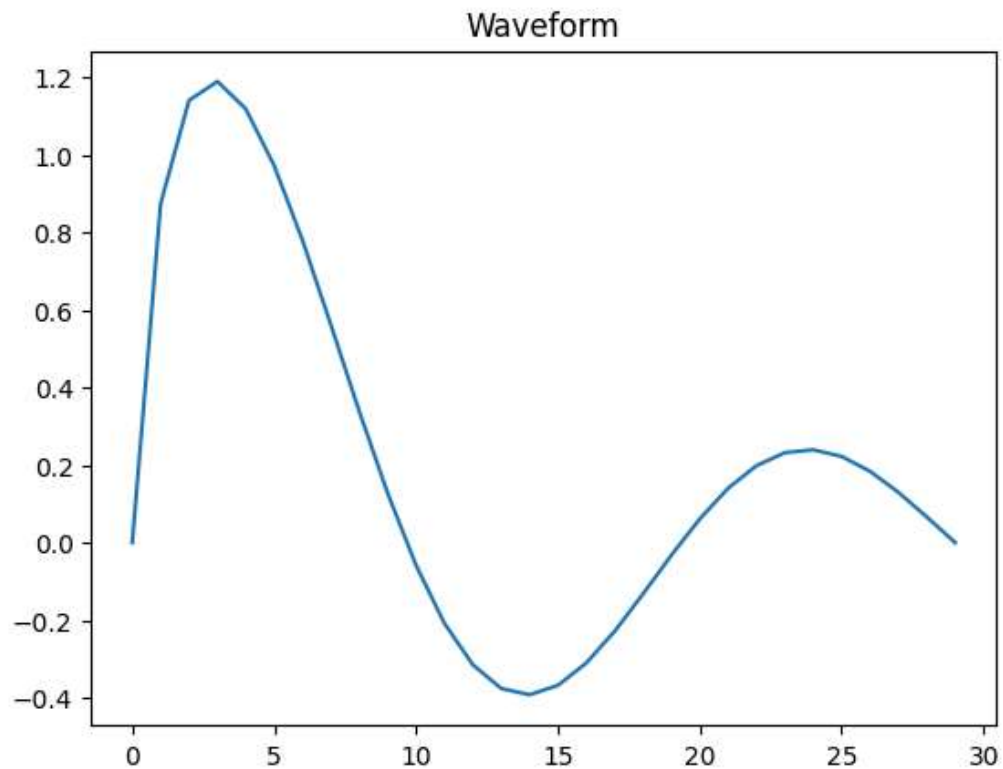
for iVal in range(nTest):
    x,y=wfDataset.__getitem__(1)
    testX=torch.cat((testX,x),0)
    testy=torch.cat((testy,y),0)

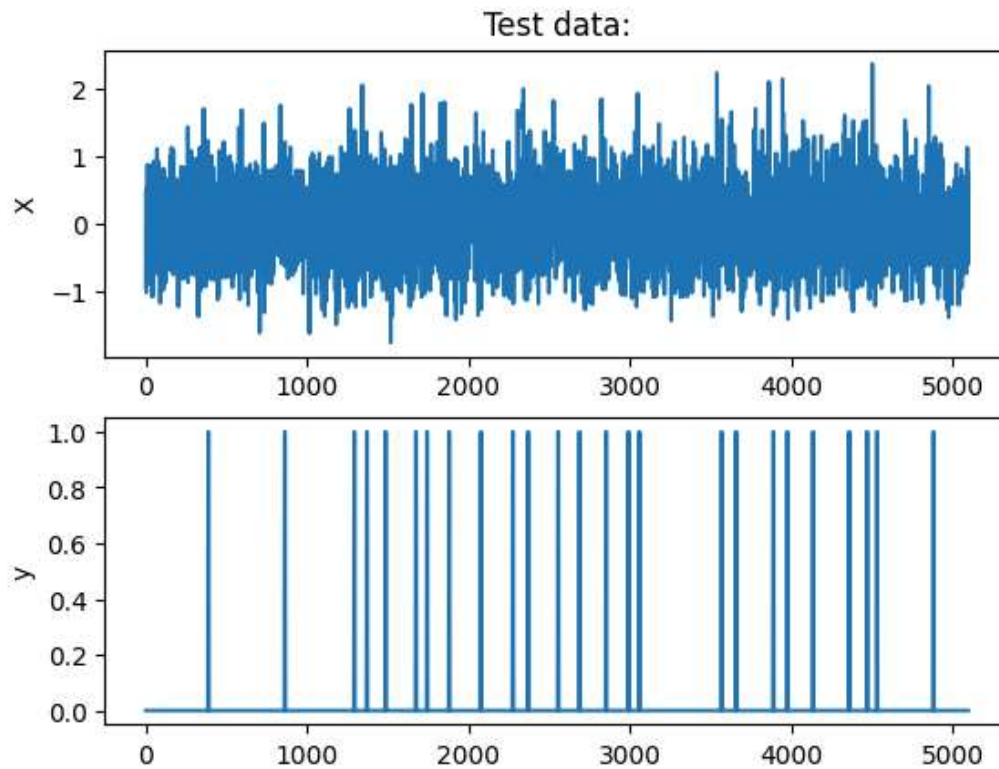
```

```
testX=torch.unsqueeze(testX,0)

plt.plot(waveform[0,:])
plt.title('Waveform')
plt.figure()
plt.subplot(2,1,1)
plt.plot(testX[0,:])
plt.ylabel('X')
plt.title('Test data:')

plt.subplot(2,1,2)
plt.plot(testy[:,0])
plt.ylabel('y')
plt.show()
```





```
In [9]: # Using LSTM to detect waveforms
class WaveformDetector(nn.Module):
    def __init__(self, input_size=1, hidden_size=16, num_layers=1, output_size=1):
        super(WaveformDetector, self).__init__()
        self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True)
        self.linear = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        #LSTM Layer
        out, _ = self.lstm(x)
        #Linear Layer
        out = self.linear(out)
        return out

# Create the model
model = WaveformDetector()
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01) # Use Adam optimizer starting rate
scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=10, gamma=0.9) # Reduce learning rate by 0.9 every 10 epochs

# %% Training Loop
num_epochs = 250
loss_list = [] # Store loss values for plotting
for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0

    for samples, labels in waveformLoader:
        # Ensure `samples` has shape (batch_size, sequence_length, input_size=1)
        samples = samples.view(samples.size(0), samples.size(1), -1) # Reshape to 3D

        # Ensure `labels` has shape (batch_size, sequence_length, output_size=1)
        labels = labels.view(labels.size(0), labels.size(1), -1) # Reshape to 3D

        # Forward pass
        outputs = model(samples)
        loss = criterion(outputs, labels)

        # Backward pass and optimization
        optimizer.zero_grad()
```

```

        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    # Reduce Learning rate
    scheduler.step()
    # Calculate average loss over all batches
    avg_loss = running_loss / len(waveformLoader)
    # Store loss for plotting
    loss_list.append(avg_loss)
    #Only print every 10 epochs
    if epoch % 25 == 24:
        print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {avg_loss:.4f}')

# Test the model
model.eval()
with torch.no_grad():
    testX = testX.view(testX.size(0), testX.size(1), -1) # Reshape to 3D

    predictions = model(testX)
    predictions = predictions.squeeze().numpy() # Convert predictions to numpy for plotting

```

```

Epoch [25/250], Loss: 0.0060
Epoch [50/250], Loss: 0.0035
Epoch [75/250], Loss: 0.0035
Epoch [100/250], Loss: 0.0036
Epoch [125/250], Loss: 0.0033
Epoch [150/250], Loss: 0.0029
Epoch [175/250], Loss: 0.0031
Epoch [200/250], Loss: 0.0032
Epoch [225/250], Loss: 0.0030
Epoch [250/250], Loss: 0.0030

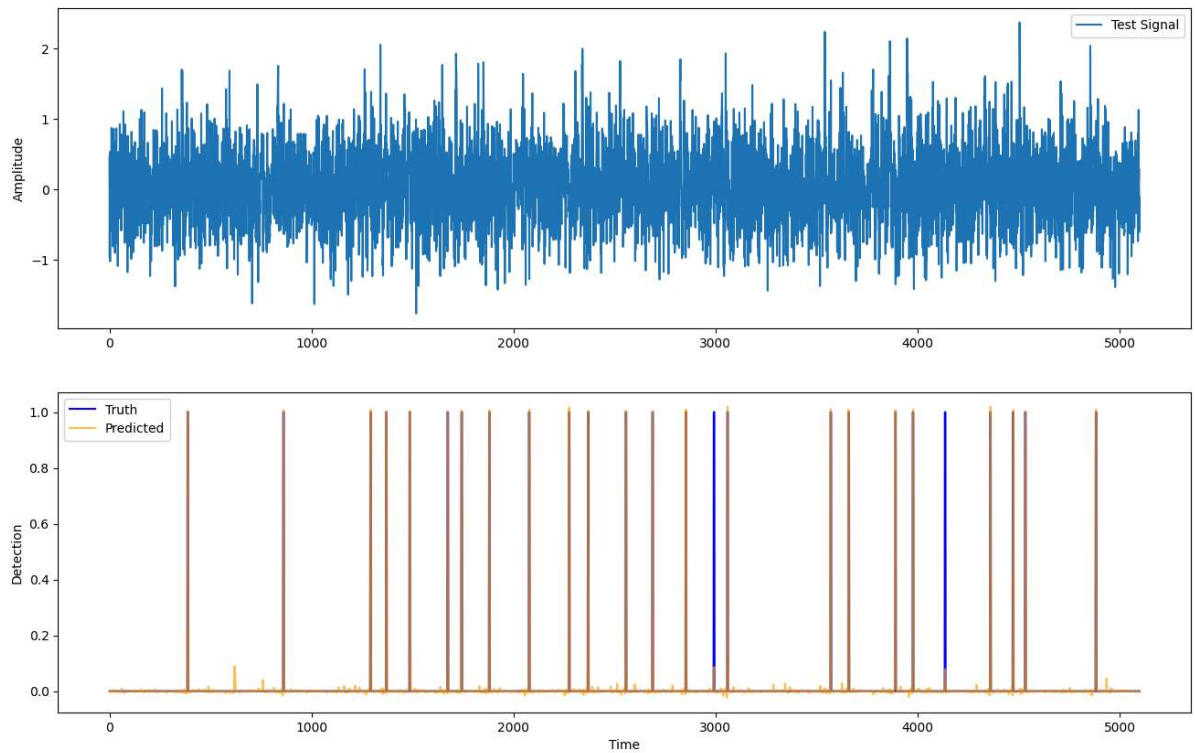
```

```

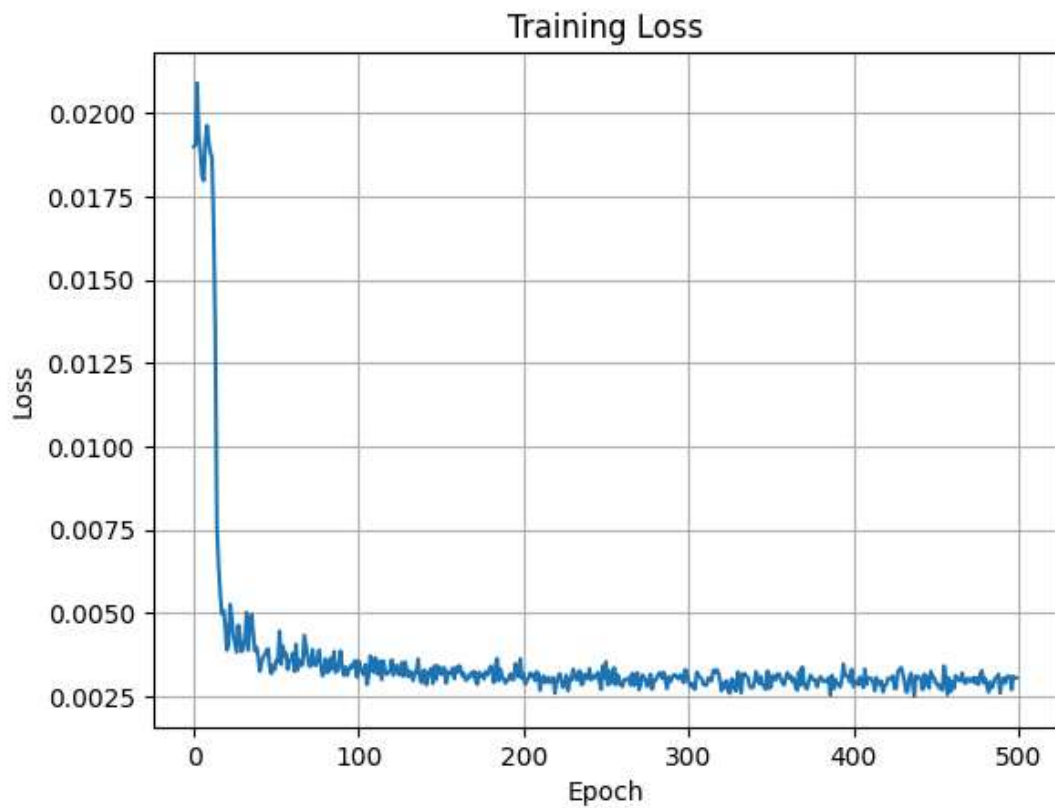
In [10]: # Plot Results
plt.figure(figsize=(16, 10))
plt.subplot(2, 1, 1)
plt.plot(testX.squeeze().numpy(), label='Test Signal')
plt.ylabel('Amplitude')
plt.legend()

plt.subplot(2, 1, 2)
plt.plot(testy.squeeze().numpy(), label='Truth', color='blue')
plt.plot(predictions, label='Predicted', color='orange', linestyle='--', alpha=0.7)
plt.xlabel('Time')
plt.ylabel('Detection')
plt.legend()
plt.show()

```



```
In [7]: plt.plot(loss_list)
plt.title('Training Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.grid()
plt.show()
```



I have chosen to use the LSTM, to make my wave detector. I have tried to use as few layers as possible. During training i have used a rather high learning rate, in order to make the algorithm converge faster. Every 10th epoch, i update the learning rate to 90% of the current value This should take care of the finetuning.

I wanted to use a binary loss function, but i struggled to obtain a good result.

Overall the net seems to have a high performance, and the loss seem to get better with higher epochs. I did not implement a breaking point to stop the training, i had faith that it would have converged with 250 epochs